

HPC Assignment 06: Parallel Interpolation

Aalok Thakkar 202301429 / Khushi Pandya 202301406

April 19, 2026

Abstract

This report presents the parallelization and performance analysis of the scatter interpolation algorithm using OpenMP. The study evaluates execution time, speedup, and scalability across five configurations on both a Lab PC and an HPC cluster. The results highlight synchronization overhead, memory bottlenecks, and scalability limitations in shared-memory systems.

1 Implementation Approach

1.1 Overview

The interpolation process maps scattered particle values to a structured grid using bilinear interpolation.

1.2 Parallelization Strategy

Particle Decomposition:

- Each thread processes a subset of particles
- Implemented using OpenMP parallel for

Race Condition Handling:

- Multiple particles update the same grid points
- Atomic operations are used to ensure correctness

1.3 Pseudocode

```
#pragma omp parallel for  
for each particle:  
  compute weights  
  update grid (atomic)
```

2 Performance Metrics

2.1 Speedup

$$S(p) = \frac{T(2)}{T(p)} \quad (1)$$

2.2 Parallel Efficiency

$$E(p) = \frac{S(p)}{p} \quad (2)$$

3 Results and Analysis

3.1 Execution Time vs Cores

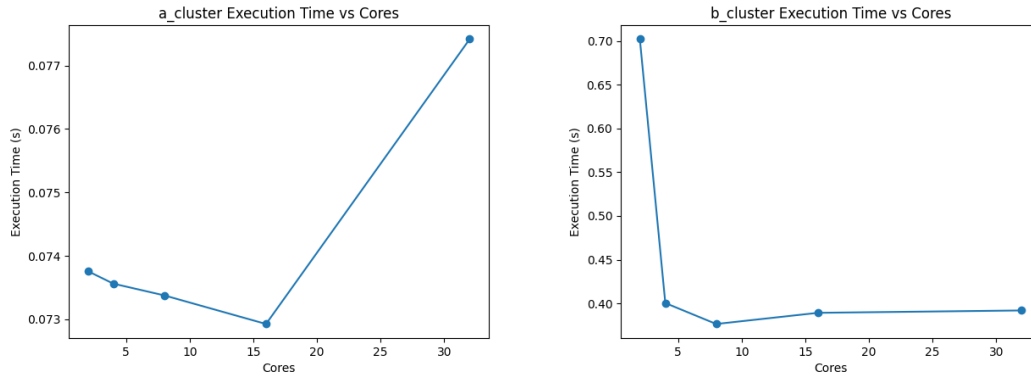


Figure 1: Execution Time vs Cores (Configs A and B - Cluster)

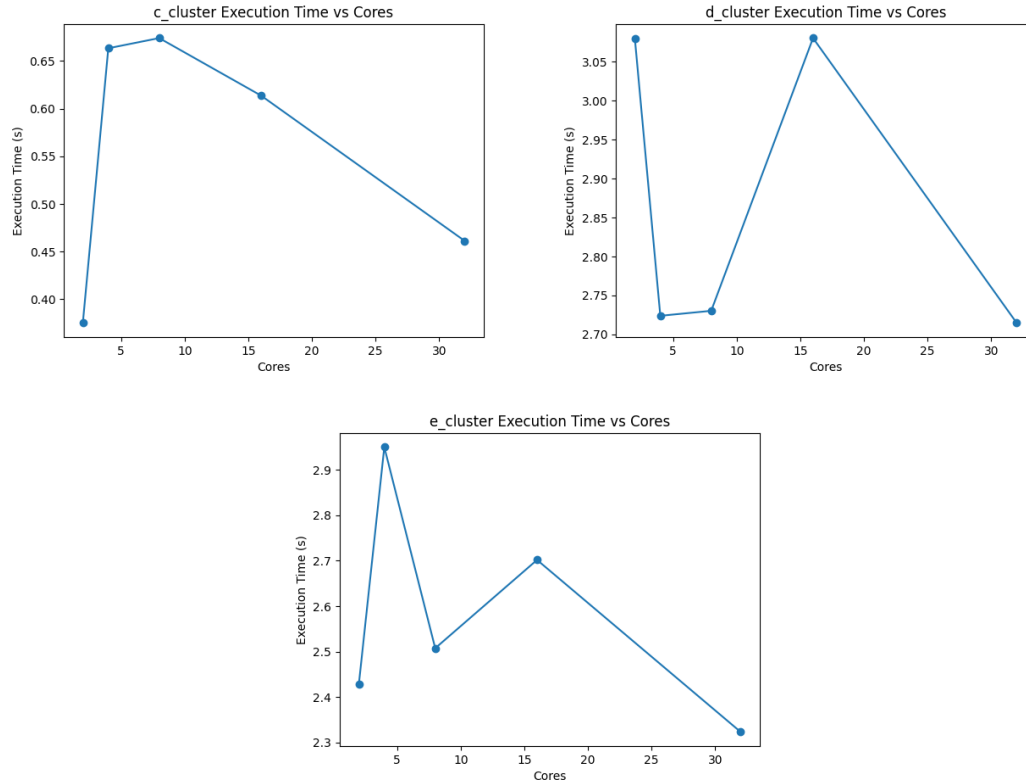


Figure 2: Execution Time vs Cores (Configs C, D, E - Cluster)

Observation:

- Execution time generally decreases with cores initially
- Some configurations show fluctuations due to overhead
- Larger workloads (D, E) show more stable scaling

3.2 Speedup vs Cores

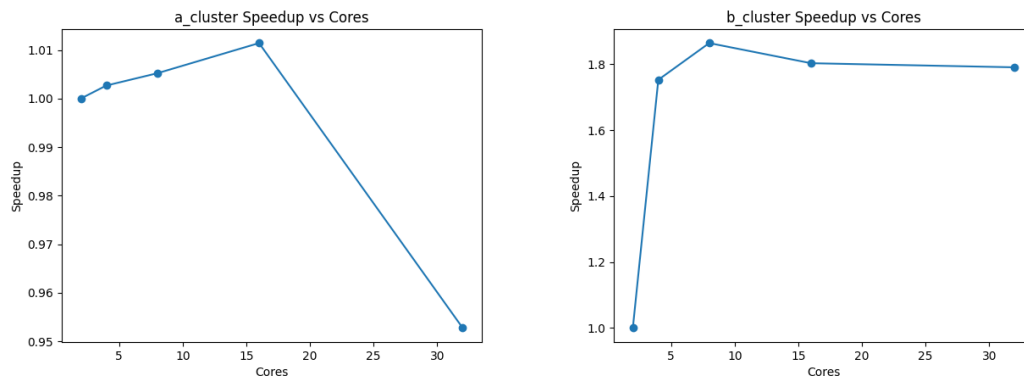


Figure 3: Speedup vs Cores (Configs A and B - Cluster)

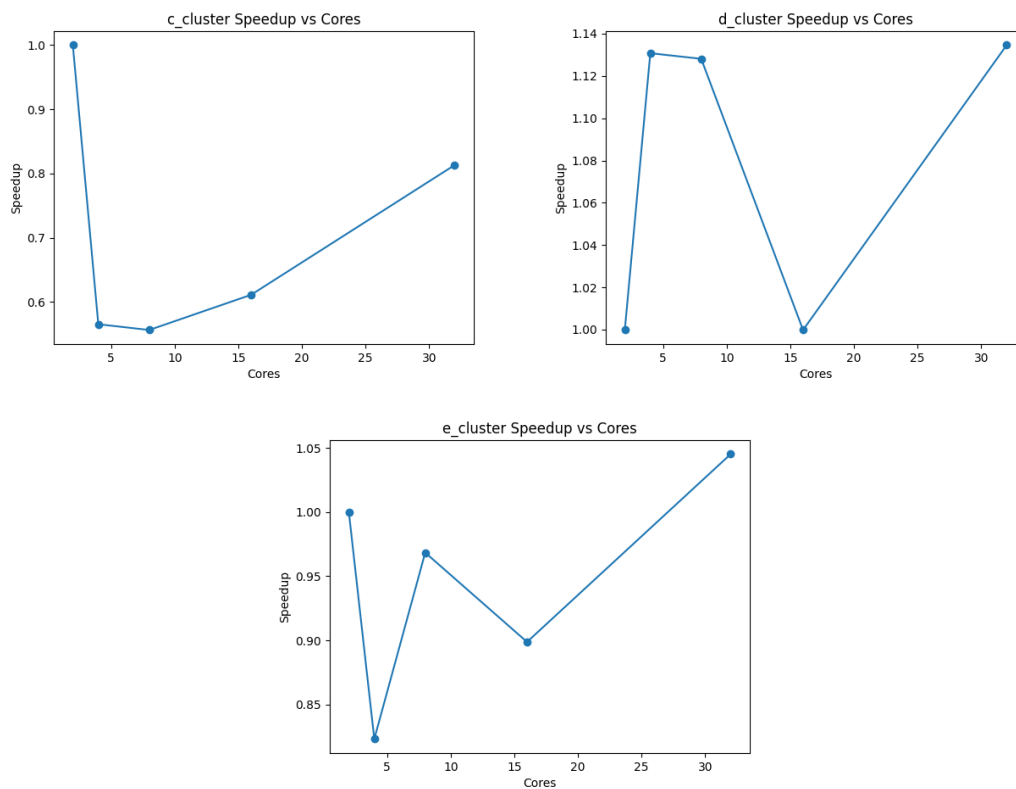


Figure 4: Speedup vs Cores (Configs C, D, E - Cluster)

Observation:

- Near-linear speedup at lower cores
- Speedup saturates at higher cores
- Some cases show degradation due to synchronization overhead

4 Parallel Efficiency and Scalability

Efficiency decreases with increasing cores due to:

- Atomic synchronization overhead
- Memory bandwidth limitations
- Cache contention

Larger configurations (D and E) scale better due to higher computation-to-overhead ratio.

5 Performance Comparison

- Maximum speedup observed ≈ 1.75
- Best performance achieved in medium-sized workloads
- Small workloads suffer from parallel overhead

6 Bottleneck Analysis

Interpolation Phase:

- Write-heavy
- Requires atomic operations
- Main bottleneck

Conclusion: The interpolation step dominates execution time and limits scalability.

7 Load Imbalance

- Uneven particle distribution leads to imbalance
- Some threads perform more updates

8 Optimization Strategies

- Grid privatization with reduction
- Spatial binning of particles
- Reduce atomic operations
- Improve memory locality

9 Conclusion

The parallel implementation improves execution time, especially for larger datasets. However, scalability is limited due to synchronization overhead and memory constraints. Future improvements should focus on reducing contention and improving data locality.